

SIMATS ENGINEERING ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE:** CSA17

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.(BL3)

Assessment Tool Weightage: 35%

QUESTIONS: 3 **Duration :** 1 Hour

Total Marks: 30

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

I Eshwanth C 192321062 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

Eshwanth C

Annexure A

Constraint-Based Problem Solving

1. A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) such that:

Constraints:

- i. D1 cannot work Night shift
- ii. D2 must work before D3 (Morning < Afternoon < Night)
- iii. Only one doctor per shift
- iv. D3 cannot work Morning
- v. All doctors must be assigned exactly one shift

Apply Backtracking Search to find a valid assignment with all intermediate steps and constraint checks. State the final valid schedule

Answer:

CSP formulation

- **Variables:** D1, D2, D3 (the shift assigned to each doctor).
- **Domain:** each variable takes a value from {M, A, N} = {Morning, Afternoon, Night}, with order $M < A < N$.
- **Constraints:** C1: $D1 \neq N$. C2: $\text{shift}(D2) < \text{shift}(D3)$. C3 (one doctor per shift) and C5 (every doctor exactly one shift): D1, D2, D3 take all-different values. C4: $D3 \neq M$.
- **Backtracking settings:** variable order $D1 \rightarrow D2 \rightarrow D3$; value order $M \rightarrow A \rightarrow N$; after every assignment all constraints whose variables are assigned are checked; on a violation the value is rejected and the next value is tried (backtrack when the domain is exhausted).

Step-by-step backtracking trace

Step	Assignment tried	Constraint checks	Result
1	D1 = M	C1: $M \neq N$ OK (no other constraint has all its variables assigned yet)	Accept $\rightarrow$ go to D2
2	D1 = M, D2 = M	C3/C5 all-different: $D2 = D1 = M$ violated	Reject ; try next value
3	D1 = M, D2 = A	C1 OK; all-different (M, A) OK; C2, C4 not yet checkable	Accept $\rightarrow$ go to D3
4	D1 = M, D2 = A, D3 = M	C4: $D3 \neq M$ violated (also all-different violated since $D1 = M$)	Reject ; try next value
5	D1 = M, D2 = A, D3 = A	all-different: $D3 = D2 = A$ violated (C2: $A < A$ also violated)	Reject ; try next value
6	D1 = M, D2 = A, D3 = N	C1: $M \neq N$ OK; C2: $A < N$ OK; C3/C5: M, A, N all different OK; C4: $N \neq M$ OK	All constraints satisfied – SOLUTION

Search tree explored (violated = constraint broken, OK = consistent so far)

```
Root (no assignment)
|-- D1 = M  OK
|   |-- D2 = M  violated (all-different)
|   |-- D2 = A  OK
|       |-- D3 = M  violated (C4)
|       |-- D3 = A  violated (all-different)
|       '-- D3 = N  OK  *** SOLUTION 1: D1=M, D2=A, D3=N ***
|           '-- D2 = N  ... (not needed; search stops at first solution)
|-- D1 = A  ... (continuing the search finds solution 2: D1=A, D2=M, D3=N)
'-- D1 = N  violated (C1)
```

Remark: if the search is continued after Solution 1, it also finds $D1 = A, D2 = M, D3 = N$ ($D1 = N$ is rejected immediately by C1; $D2 = N$ or $D3 = A$ never satisfy C2). So the problem has exactly **two** valid schedules, and C4 + C2 force D3 to work the Night shift. Applying node consistency first ($D1$ in $\{M, A\}$, $D3$ in $\{A, N\}$) would prune the tree even more.

Final valid schedule

Shift	Morning	Afternoon	Night
Doctor	D1	D2	D3

Result: $D1 \rightarrow \text{Morning}, D2 \rightarrow \text{Afternoon}, D3 \rightarrow \text{Night}$ (all five constraints satisfied; the alternative valid schedule is $D2 \rightarrow \text{Morning}, D1 \rightarrow \text{Afternoon}, D3 \rightarrow \text{Night}$).

2. A robot operates in a grid environment (5×5). It must move from Start (S) to Goal (G). Obstacles block certain paths.

Constraints:

- Movement allowed: Up, Down, Left, Right
- Cost of each move = 1
- Cannot pass through obstacles
- Use Manhattan Distance heuristic

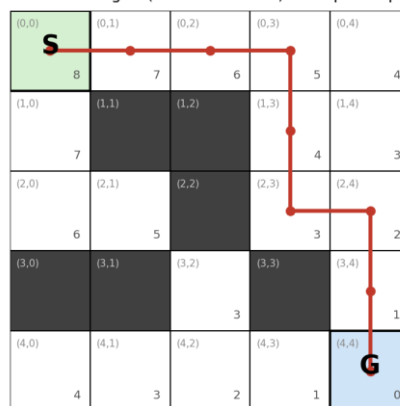
Using above constraints and BFS Search Algorithm Show step-by-step node expansion and priority queue updates and determine the optimal path and cost.

Answer:

Problem understanding and assumed grid

The question does not include the grid picture, so the following 5×5 grid is assumed (cells written as (row, column), row 0 at the top, column 0 at the left): **Start S = (0,0)**, **Goal G = (4,4)**, **obstacles = (1,1), (1,2), (2,2), (3,0), (3,1), (3,3)**. The small number in the lower-right corner of each free cell is its Manhattan distance $h(n) = |r - 4| + |c - 4|$ to G. The same steps can be repeated for any other grid.

Assumed 5×5 grid (dark = obstacle) and optimal path



Method selection

- All moves cost 1 (uniform cost), so BFS is guaranteed to find the optimal (shortest) path. BFS expands the nodes level by level using a FIFO queue.
- Because a Manhattan-distance heuristic and a priority queue are required, the frontier is ordered by $f(n) = g(n) + h(n)$ (best-first with path cost = A^*). h is admissible and consistent (every move changes h by at most 1), so the first time G is removed from the queue the path is optimal.
- Tie-breaking: lowest f , then lowest h , then first-generated. Successor order: Up, Down, Left, Right. A closed list avoids re-expanding nodes.

Step-by-step node expansion and priority-queue updates

Step	Node expanded (g, h, f)	Children generated / updated (node: g, h, f)	Priority queue after the step (node: f)
0	–	Initial: S(0,0) g=0, h=8, f=8	(0,0):8
1	(0,0) (0, 8, 8)	(1,0): 1, 7, 8; (0,1): 1, 7, 8	(1,0):8, (0,1):8
2	(1,0) (1, 7, 8)	(2,0): 2, 6, 8	(2,0):8, (0,1):8
3	(2,0) (2, 6, 8)	(2,1): 3, 5, 8	(2,1):8, (0,1):8
4	(2,1) (3, 5, 8)	none (all neighbours blocked/closed)	(0,1):8
5	(0,1) (1, 7, 8)	(0,2): 2, 6, 8	(0,2):8
6	(0,2) (2, 6, 8)	(0,3): 3, 5, 8	(0,3):8
7	(0,3) (3, 5, 8)	(1,3): 4, 4, 8; (0,4): 4, 4, 8	(1,3):8, (0,4):8
8	(1,3) (4, 4, 8)	(2,3): 5, 3, 8; (1,4): 5, 3, 8	(2,3):8, (1,4):8, (0,4):8
9	(2,3) (5, 3, 8)	(2,4): 6, 2, 8	(2,4):8, (1,4):8, (0,4):8
10	(2,4) (6, 2, 8)	(3,4): 7, 1, 8	(3,4):8, (1,4):8, (0,4):8
11	(3,4) (7, 1, 8)	(4,4): 8, 0, 8	(4,4):8, (1,4):8, (0,4):8
12	(4,4) (8, 0, 8)	Goal test passed – STOP	–

Expansion order (closed list): (0,0) → (1,0) → (2,0) → (2,1) → (0,1) → (0,2) → (0,3) → (1,3) → (2,3) → (2,4) → (3,4) → (4,4).

Optimal path

Following parent pointers back from G gives: **(0,0) → (0,1) → (0,2) → (0,3) → (1,3) → (2,3) → (2,4) → (3,4) → (4,4)**.

Directions: Right, Right, Right, Down, Down, Right, Down, Down (8 moves).

Optimal cost = 8 (8 moves × 1). This equals the Manhattan distance from S to G ($h(S) = 8$), so no path can be shorter.

Comparison with plain BFS

Plain BFS (FIFO queue, no heuristic) expands 14 nodes before reaching G, whereas the heuristic-guided priority queue expands 12 nodes. Both return a path of cost 8; the heuristic only saves work. Time and space complexity of BFS is $O(b^d)$ with $b \leq 4$ and $d = 8$.

Result: the optimal path from S to G is shown in red in the figure above, with a total cost of 8.

3. An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The robot operates in a partially observable environment where some paths are blocked, and it must make decisions with limited information.

The building is represented as a grid of rooms. The robot starts at position S and must reach a survivor located at G.

Constraints:

- The robot can move up, down, left, right
- Some cells are blocked (obstacles) and cannot be traversed
- The robot has limited sensing capability (can only observe adjacent cells)
- Each movement has a cost of 1, but entering risky zones has an additional cost of 2
- The robot must minimize total cost while ensuring safe navigation
- The robot must update its knowledge dynamically (online search scenario)

Tasks:

- Analyze all the given constraints and explain how they affect the robot's decision-making process.
- Develop a solution approach using an appropriate search strategy (e.g., Uniform Cost Search / Online Search Agent). Clearly explain the steps involved.
- Demonstrate how the robot applies AI concepts such as: Intelligent agents, Rationality, Environment type
- Justify why your chosen approach is the most suitable for solving this problem under the given constraints.

Answer:

a) Analysis of the constraints

Constraint	Effect on the robot's decision making
Moves: up, down, left, right	Branching factor ≤ 4 ; the state space is the grid graph, and each action is a move to an adjacent room. Diagonal shortcuts are not available.
Blocked cells	Cells that can never be entered: they remove edges from the graph and can invalidate a planned path, so the plan must be repaired when a block is discovered. A path may not exist at all if blocks cut the building.
Limited sensing (adjacent cells only)	The environment is partially observable : the agent cannot plan once and trust the plan; it must keep an internal map (belief state), mark unknown cells as "unknown", and choose actions with incomplete information (online search).
Cost 1 per move, +2 for risky cells	A risky room costs $1 + 2 = 3$, so the shortest path in steps is not always the cheapest. Search must use path cost (UCS / A*), not just the number of steps (BFS) – the robot may take a longer but safer route.
Minimise total cost, safe navigation	Performance measure: lowest total cost, and blocked cells are never entered. Safety is a hard constraint (blocked cells excluded); risk is a soft constraint (priced by the extra cost of 2).
Dynamic knowledge update (online search)	The agent interleaves sense → update map → plan → act . Plans are only valid until new information arrives, so replanning is required after every observation that changes the map.

b) Solution approach – Online search agent using Uniform Cost Search (with replanning)

Idea: because the robot cannot see the whole building, it repeatedly runs UCS on its *current knowledge* of the map, executes only the first step of the best plan, senses its neighbours, and replans. Unknown cells are treated optimistically (free, cost 1) so the robot is not discouraged from exploring; known blocked cells are removed from the graph, and known risky cells cost 3. (UCS can be replaced by A* with the Manhattan heuristic, which is admissible here because each move costs at least 1.)

```
function ONLINE-RESCUE-AGENT(percept) returns an action
  persistent: map    // known blocked cells, known risky cells (unknown = assumed free)
                pos   // current cell
                goal  // survivor location G
  map <- UPDATE-MAP(map, percept)           // percept = status of the 4 adjacent cells
  if pos = goal then return STOP             // survivor reached
  plan <- UCS(map, pos, goal)               // step cost: 1, or 1+2 = 3 if cell is risky
  if plan = failure then return BACK-OFF-OR-EXPLORE-NEAREST-UNKNOWN // no known route
  return FIRST-ACTION(plan)                // execute one step only, then sense again

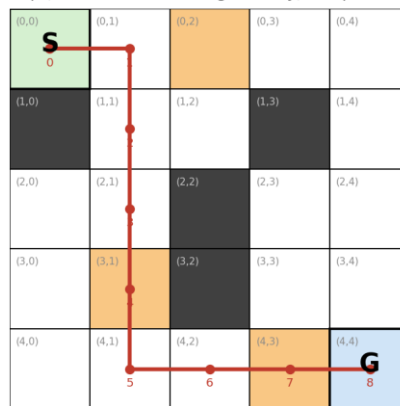
function UCS(map, start, goal)
  frontier <- priority queue ordered by g(n) = path cost; explored <- { }
  INSERT(start, g=0)
  while frontier not empty do
    n <- POP-MIN(frontier)
    if n = goal then return SOLUTION(n)
    add n to explored
    for each neighbour m of n (up, down, left, right) that is not a known blocked cell do
      cost <- g(n) + 1 + (2 if m is a known risky cell else 0)
      if m not in explored and not in frontier, or cost < g(m) then INSERT/UPDATE(m, cost)
  return failure
```

Steps involved: (1) sense the adjacent cells; (2) update the map (blocked / risky / free); (3) run UCS from the current cell to G on the known map; (4) take the first move of the cheapest plan; (5) repeat from (1) until G is reached. If no known route exists, the robot explores the nearest unknown cell.

Worked demonstration (assumed building)

Assumed true map (hidden from the robot): S = (0,0), G = (4,4); blocked cells (1,0), (1,3), (2,2), (3,2); risky cells (0,2), (3,1), (4,3). Cost of a move = 1, or 3 if the cell entered is risky.

True map (dark = blocked, orange = risky) and path travelled



t	Robot at	Newly sensed	Plan from known map (est. cost)	Move (cost)	Total so far
0	(0,0)	(1,0) blocked	(0,0) → (0,1) → (1,1) → (2,1) → (3,1) → (4,1) → (4,2) → (4,3) → (4,4) (8)	to (0,1) (1)	1
1	(0,1)	(0,2) risky	(0,1) → (1,1) → (2,1) → (3,1) → (4,1) → (4,2) → (4,3) → (4,4) (7)	to (1,1) (1)	2
2	(1,1)	–	(1,1) → (2,1) → (3,1) → (4,1) → (4,2) → (4,3) → (4,4) (6)	to (2,1) (1)	3

t	Robot at	Newly sensed	Plan from known map (est. cost)	Move (cost)	Total so far
3	(2,1)	(3,1) risky, (2,2) blocked	(2,1) → (3,1) → (4,1) → (4,2) → (4,3) → (4,4) (7)	to (3,1) (3)	6
4	(3,1)	(3,2) blocked	(3,1) → (4,1) → (4,2) → (4,3) → (4,4) (4)	to (4,1) (1)	7
5	(4,1)	–	(4,1) → (4,2) → (4,3) → (4,4) (3)	to (4,2) (1)	8
6	(4,2)	(4,3) risky	(4,2) → (4,3) → (4,4) (4)	to (4,3) (3)	11
7	(4,3)	–	(4,3) → (4,4) (1)	to (4,4) (1)	12
8	(4,4)	–	Goal reached	–	12

The robot reaches the survivor with a total cost of **12** along $(0,0) \rightarrow (0,1) \rightarrow (1,1) \rightarrow (2,1) \rightarrow (3,1) \rightarrow (4,1) \rightarrow (4,2) \rightarrow (4,3) \rightarrow (4,4)$. It replans after every new observation (e.g., at $t = 0$ the blocked cell (1,0) forces it to go right; at $t = 3$ it discovers (3,1) is risky and (2,2) is blocked, but the detour would cost more, so it accepts the risky cell at cost 3). A planner with a full map would have found a cost-10 route (via the top row, accepting the risky cell (0,2)); the extra 2 units is the unavoidable price of acting with limited information – each decision was still the cheapest according to what the robot knew at that moment.

c) Application of AI concepts

Concept	How it appears in the rescue robot
Intelligent agent	Percepts: status of the 4 adjacent cells (free / blocked / risky), own position, survivor detection. Actions: Up, Down, Left, Right, Stop. Architecture: a goal-based (utility-based) agent with internal state – it stores the map, searches for a plan, and compares costs.
Rationality	A rational agent picks the action that maximises expected performance <i>given the percept sequence and its knowledge</i> . Here, it picks the move of the cheapest known plan (lowest cost, no blocked cell, avoiding risky cells unless they are cheaper overall) and updates its beliefs. Rationality $\neq$ omniscience: the extra cost in the demonstration is not irrational.
Environment type	Partially observable (adjacent cells only); single agent ; deterministic moves (the next cell is predictable, assuming no slipping – may be stochastic in a real collapse); sequential (current moves affect later cost); static during planning (assume rooms do not change while sensing – may be dynamic with aftershocks/fire); discrete grid; unknown (map must be learned online).

d) Why this approach is the most suitable

- **Online (sense–plan–act) search** is needed because the environment is partially observable and unknown: an offline plan made at the start would fail when an unseen block appears. Replanning after each observation handles this.
- **UCS / A* handle step costs.** BFS and DFS ignore the +2 risk cost, and greedy search ignores $g(n)$; UCS is complete and optimal for non-negative costs on the known map, so each plan is the minimum-cost route given current knowledge.
- **Optimistic assumption for unknown cells** keeps the robot moving toward G instead of freezing, and any wrong guess is repaired by sensing and replanning; with known blocked cells removed from the graph, it never enters blocked rooms (safety).
- **Low cost of replanning:** a 5×5 grid has at most 25 nodes, so rerunning UCS after each step is cheap, and the robot can reuse its map memory. In larger buildings, A* with Manhattan distance (admissible) or LRTA*, which learns heuristic values, reduces the search effort.
- **Alternatives are weaker:** pure reflex agents may loop; random exploration is wasteful; offline UCS with a full map is not possible without full knowledge.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided